MANUAL DEL PROGRAMADOR

Juego de Naves en Consola

Actividad 8 — Programación Orientada a Objetos en C++

Autor: Alma Olivola
Institución: E.E.S.T. N.º 1 — Vicente López
Materia: Laboratorio de Programación (LPR)
Curso: 5.º año
Versión: 1.0.0
Fecha: 22 de octubre de 2026

1. Introducción

Este manual explica la estructura y el funcionamiento interno del proyecto Juego de Naves en Consola, desarrollado en C++ para la Actividad 8 de Laboratorio de Programación.

El objetivo es describir cómo está organizado el código, qué funciones cumplen las clases principales y cómo compilar, ejecutar y modificar el programa.

2. Tecnologías utilizadas

* C++: lenguaje utilizado para desarrollar el juego.
* Visual Studio Code: editor utilizado para escribir y organizar el código.
* g++ (MinGW): compilador para generar el archivo ejecutable.
* Git: sistema de control de versiones para registrar los cambios del proyecto.

3. Estructura del proyecto

El proyecto se organiza en carpetas y archivos para facilitar su mantenimiento.

```
Juego-de-naves-en-consola/
├── .gitattributes
├── .gitignore
├── LICENSE
├── README.md
├── CHANGELOG.md
├── docs/
│   └── manuales/
│       ├── manual_programador_v1.0.0.pdf
│       └── manual_usuario_v1.0.0.pdf
├── src/
│   └── main.cpp
└── capturas/
    └── combate_consola.png
```

* src/main.cpp: contiene el código fuente del juego.
* docs/: contiene la documentación del proyecto.
* capturas/: contiene imágenes del juego en funcionamiento.
* .gitignore: indica qué archivos debe ignorar Git.
* .gitattributes: configura cómo Git trata determinados archivos.
* LICENSE: establece las condiciones de uso del código.
* README.md: presenta el proyecto y explica cómo ejecutarlo.
* CHANGELOG.md: registra los cambios realizados.

4. Programación Orientada a Objetos

El proyecto utiliza Programación Orientada a Objetos (POO), un enfoque que permite organizar el programa mediante clases y objetos.

4.1. Clase Nave

La clase Nave representa al jugador dentro del juego. Contiene los datos necesarios para identificar y controlar la nave, como su posición y sus vidas.

Sus responsabilidades principales son:

* Representar la nave del jugador.
* Guardar sus datos.
* Permitir su movimiento.
* Gestionar las acciones que puede realizar.

4.2. Clase Asteroide

La clase Asteroide representa los obstáculos que aparecen durante la partida.

Sus responsabilidades son:

* Representar cada asteroide.
* Mantener su posición.
* Actualizar su movimiento durante el juego.

4.3. Clase Bala

La clase Bala representa los disparos realizados por el jugador.

Sus responsabilidades son:

* Representar un proyectil.
* Controlar su movimiento.
* Permitir que participe en la mecánica de disparo.

Nota: las clases y responsabilidades anteriores describen el diseño general del juego. Los atributos y métodos exactos deben coincidir con los implementados en main.cpp.

5. Encapsulamiento: private y public

El encapsulamiento permite proteger los datos internos de una clase.

* private: permite acceder a los miembros desde la propia clase y desde los lugares autorizados por C++.
* public: permite utilizar desde fuera de la clase los miembros que se hayan declarado públicos.

Por ejemplo, mantener las vidas de la nave como un atributo privado evita que otra parte del programa las modifique directamente sin control. Para cambiar ese valor, se pueden utilizar métodos públicos que comprueben que la operación sea válida.

6. Constructor de la clase Nave

Un constructor es una función especial que se ejecuta automáticamente al crear un objeto.

Por ejemplo:

Nave(int _x, int _y, int _vidas);

Este constructor recibe valores para inicializar la posición y las vidas de la nave. Así, el objeto comienza con los datos necesarios para participar en el juego.

Los parámetros concretos deben coincidir con el constructor utilizado en el código fuente.

7. Funcionamiento general del programa

El funcionamiento previsto del juego es el siguiente:

1. Se inicia el programa.
2. Se muestra el menú principal.
3. El usuario selecciona una opción.
4. Si elige jugar, controla la nave mediante el teclado.
5. Puede desplazarse y disparar para enfrentarse a los asteroides.
6. El programa actualiza el estado de la partida y la puntuación, si estas funciones están implementadas.
7. La partida termina cuando se cumple la condición de finalización definida en el código.

8. Funcionalidad adicional

El proyecto incorpora como mejora prevista un sistema de puntuación y dificultad progresiva.

El sistema de puntuación permite registrar los puntos obtenidos por el jugador. La dificultad progresiva consiste en aumentar el desafío durante la partida, por ejemplo, incrementando la velocidad de los asteroides.

Para que esta funcionalidad esté completa, el código debe actualizar la puntuación cuando corresponda y modificar la dificultad durante la ejecución.

Importante: se debe verificar que esta mejora coincida con una de las 15 opciones del anexo de la actividad y que esté implementada en el código entregado.

9. Compilación y ejecución

Para compilar el programa, abrir la terminal de VS Code y entrar en la carpeta src:

cd src

Ejecutar el siguiente comando:

g++ main.cpp -o Juego-de-naves-en-consola.exe

Si la compilación finaliza sin errores, ejecutar el programa desde PowerShell:

.\Juego-de-naves-en-consola.exe

Si aparecen errores, revisar el código, la instalación del compilador y la ubicación de los archivos.

10. Mantenimiento del código

Para modificar el juego de manera organizada, se recomienda:

* Realizar una copia de seguridad antes de cambios importantes.
* Utilizar nombres claros para las variables, funciones y clases.
* Mantener la indentación y los comentarios del código.
* Probar el programa después de cada modificación.
* Registrar los cambios importantes en CHANGELOG.md.
* Guardar los avances mediante Git.
* Actualizar los manuales cuando cambie el funcionamiento del programa.

11. Control de versiones

Git permite registrar los cambios realizados durante el desarrollo. Cada modificación importante puede guardarse en un commit con un mensaje que explique qué se cambió.

Ejemplo:

git add .
git commit -m "docs: agregar manual del programador"

Esto facilita consultar el historial y recuperar versiones anteriores del proyecto.

12. Conclusión

El proyecto permite aplicar los conceptos básicos de Programación Orientada a Objetos en C++, mediante clases, objetos, constructores y encapsulamiento. También permite practicar la organización de archivos, la documentación técnica, la compilación y el uso de Git para mantener un historial de los cambios.

Este manual sirve como referencia para comprender y mantener el código fuente del Juego de Naves en Consola.